

GenTA Competency Assessment

Mini GACS Prototype — Mood & Style Embedding Pipeline for Art/Marketing Videos

Author: Nabanshu Barman

Date: 27 February 2026

1. Problem Framing in GenTA Context

GenTA's vision centers around building a GACS (GenTA Affective Computing System) engine capable of understanding the emotional and stylistic “feel” of contemporary art and marketing creatives. In digital marketing and visual storytelling, performance is not determined solely by objects in an image, but by tone, color palette, composition, emotional expression, and aesthetic coherence.

This prototype aims to demonstrate a minimal but rigorous version of such a system. Specifically, it implements an end-to-end pipeline that:

- Extracts representative frames from short art/marketing-style videos.
- Computes fixed-size image embeddings using a pre-trained multimodal vision model.
- Measures cosine similarity between embeddings to estimate mood/style similarity.
- Visualizes embedding relationships to interpret clustering and “vibe” cohesion.

The goal is not large-scale training, but to show how a verification-first AI R&D workflow can produce a reproducible, extensible affective embedding system that could evolve into a production-grade GACS engine.

2. Dataset Description & Licensing

2.1 Video Selection Strategy

Three short (~30 seconds each), royalty-free videos were selected to represent distinct aesthetic and emotional profiles:

1. Emotion-Dominant Visual

- Close-up of a woman expressing sadness against a red background.
- Strong emotional signal, minimal scene variation.

2. Style-Dominant Visual (Art Gallery Pan)

- Camera panning over multiple artworks.
- Diverse textures and color palettes.

3. High-Energy Visual (Basketball Clip)

- Outdoor sports footage.
- Dynamic motion and natural lighting.

The selection intentionally introduced contrast across:

- Emotional intensity
- Stylistic diversity
- Motion dynamics

This contrast allows the embedding space to reveal intra-video cohesion and inter-video separation.

2.2 Licensing

All videos were sourced from royalty-free or public-domain repositories (e.g., Pexels / Pixabay).

Licenses were verified to allow redistribution or usage without attribution requirements.

Video URLs and license details are documented in DATA_LICENSES.md.

3. System Architecture Overview

The pipeline is modular and CLI-driven. It consists of three core stages:

3.1 Frame Extraction

Script: `src/extract_frames.py`

- Interval-based sampling (e.g., 1 frame every 2–3 seconds).
- Saves frames under:
 - `data/frames/<video_id>/`
- Produces metadata CSV including:
 - `video_id`
 - `timestamp_s`
 - `file_path`
 - `width / height`
 - `md5 hash`

Each video yields 20–50 representative frames.

3.2 Embedding Computation

Script: `src/compute_embeddings.py`

Model used:

`openai/clip-vit-base-patch32`

Rationale:

- Strong semantic + stylistic embeddings
- Lightweight and CPU-compatible
- Easy integration via HuggingFace

Pipeline features:

- Batch processing
- Device detection (CPU/GPU)
- L2 normalization
- Outputs:
 - `outputs/embeddings.npy`
 - `outputs/embeddings_index.csv`

Embeddings shape:

`(N_frames, 512)`

3.3 Similarity & Visualization

Script: `src/similarity.py`

Steps:

1. Normalize embeddings.
2. Compute cosine similarity:
3. $S = emb @ emb.T$
4. Retrieve top-5 neighbors per query.
5. Generate:
 - `heatmap.png`
 - `tsne.png`
 - `topk.csv`

- Query-neighbor image grids

4. Verification & Testing Strategy

A verification-first approach was enforced at each stage.

4.1 Embedding Verification

Checks implemented:

- Shape correctness (N, 512)
- No NaNs:
 - `assert not np.isnan(embeddings).any()`
- L2 normalization validation:
 - `norms ≈ 1.0`
- Duplicate image → identical embedding (within tolerance)

4.2 Similarity Verification

- Symmetry check:
 - `np.allclose(S, S.T)`
- Diagonal values ≈ 1
- Similarity values clipped to $[-1, 1]$

4.3 Automated Tests

Executed via:

```
pytest -q
```

Test coverage includes:

- `test_no_nans`
- `test_embedding_dimensions`
- `test_duplicate_images_equal_embeddings`

All tests passed before final submission.

5. Results & Interpretation

5.1 Heatmap Analysis

The pairwise cosine similarity heatmap shows:

- High similarity blocks within individual videos.
- Lower similarity across different video categories.

This confirms intra-video mood/style cohesion.

5.2 t-SNE Visualization

The t-SNE plot reveals:

- Distinct clusters corresponding to video categories.
- Emotion-focused frames cluster tightly.
- Art gallery frames show moderate spread (stylistic variation).
- Basketball frames cluster distinctly from emotional frames.

5.3 Query + Top-5 Neighbor Retrieval

For selected query frames:

- Crying face queries retrieve similar emotional close-ups.
- Basketball queries retrieve motion-consistent frames.
- Art queries retrieve stylistically related artworks.

This demonstrates that CLIP embeddings capture not just object semantics, but aesthetic consistency.

6. AI Assistant Usage & Governance

6.1 Tools Used

- GitHub Copilot (primary)
- ChatGPT (secondary, conceptual clarification)

6.2 Scope of AI Assistance

AI was used for:

- Boilerplate CLI parsing
- Batch-processing scaffolds
- Plotting snippets
- Minor debugging suggestions

AI was NOT used to:

- Design architecture

- Define verification strategy
- Choose evaluation metrics
- Interpret results

6.3 Oversight & Validation

All AI-generated snippets were:

- Reviewed manually
- Adapted to match architectural design
- Tested via assertions and pytest
- Verified through output inspection

Detailed logs are available in `ai_assistant_log.md`.

7. Reproducibility & Runtime

Environment:

- Python 3.9+
- CPU-compatible
- Runtime < 15 minutes on CPU

Repro steps:

1. Install dependencies
2. Extract frames
3. Compute embeddings
4. Run similarity script
5. Open notebook for inspection

All outputs saved under `outputs/`.

8. Extension Toward Full GACS Engine

This prototype can evolve into a production GACS system via:

8.1 Multimodal Fusion

- Combine image + audio embeddings
- Add OCR/text embeddings for creative copy

- Fuse modalities for richer vibe estimation

8.2 Performance Calibration (CTR / ROAS)

1. Collect historical CTR/ROAS per creative.
2. Train regression model:
3. embedding → predicted CTR
4. Rank creatives by predicted performance.
5. Use A/B testing to validate uplift.

8.3 Production Architecture

- Vector database (FAISS / Milvus)
- API for real-time similarity queries
- Creative upload → vibe score → ranking

8.4 Human-in-the-Loop Feedback

- Collect human vibe labels
- Fine-tune embedding model
- Improve sensitivity to emotional nuance

9. Conclusion

This Mini GACS Prototype demonstrates:

- A reproducible embedding-based affective computing pipeline
- Verification-first AI engineering discipline
- Clear architectural modularity
- Meaningful similarity behavior aligned with perceived mood/style
- Strong foundation for extending toward marketing performance integration

The system successfully shows that pre-trained multimodal embeddings can serve as a viable foundation for estimating aesthetic and emotional similarity across marketing creatives — aligning directly with GenTA's mission of making contemporary art and visual media emotionally interpretable.